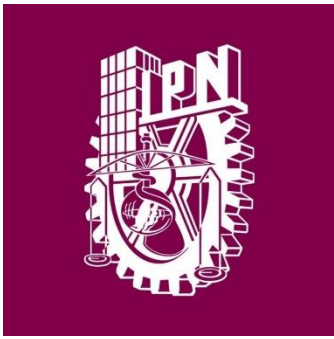




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 8

Integrantes:

Vázquez Blancas Cesar Said

Diaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martin Rafael

Introducción Teórica

En los sistemas operativos, la sincronización de procesos es un concepto fundamental que permite coordinar la ejecución de múltiples procesos o hilos para garantizar el acceso ordenado y seguro a recursos compartidos. Los mecanismos de sincronización son esenciales en entornos donde múltiples procesos cooperativos interactúan entre sí, ya que estos deben evitar conflictos como condiciones de carrera, inconsistencias de datos o bloqueos.

Sincronización con Semáforos

Un semáforo es una herramienta ampliamente utilizada para controlar el acceso a recursos compartidos en sistemas de programación concurrente. Fue introducido por Edsger Dijkstra y se basa en un contador entero que restringe el número de procesos que pueden acceder simultáneamente a un recurso crítico.

Los semáforos pueden clasificarse en:

1. **Semáforos Binarios:** Funcionan como un interruptor (0 o 1), donde un proceso puede bloquear o liberar el recurso.
2. **Semáforos Contadores:** Permiten que múltiples procesos accedan a un recurso limitado, utilizando un contador que decrece al ocupar un recurso y aumenta al liberarlo.

Funcionamiento

- Cuando un proceso intenta acceder a un recurso protegido por un semáforo:
 - Si el semáforo tiene un valor mayor a 0, el proceso accede al recurso y decrementa el valor del semáforo.
 - Si el semáforo tiene un valor de 0, el proceso se bloquea hasta que otro proceso libere el recurso y aumente el valor del semáforo.
- Este mecanismo asegura la exclusión mutua, donde solo un proceso (o un número permitido de procesos) puede acceder a un recurso crítico a la vez.

Semáforos en Linux y Windows

1. En Linux:

- Linux implementa semáforos utilizando bibliotecas como `semaphore.h` en C. Los semáforos pueden ser utilizados para la sincronización entre procesos diferentes o entre hilos de un mismo proceso.
- Ejemplo: Funciones como `sem_init`, `sem_wait`, y `sem_post` son comunes para inicializar, bloquear y desbloquear un semáforo.

2. En Windows:

- Windows proporciona semáforos a través de su API de Win32. Los desarrolladores pueden utilizar funciones como `CreateSemaphore`, `WaitForSingleObject`, y `ReleaseSemaphore`.
- Estas funciones permiten manejar el acceso a recursos compartidos tanto en aplicaciones monohilo como en sistemas multihilo.

1. A través de la ayuda en línea que proporciona Linux, investigue el funcionamiento de las funciones: `semget()`, `semop()`. Explique los argumentos, retorno de las funciones y las estructuras relacionadas con dichas funciones.

1. Función `semget()`

En Linux, los semáforos se manejan utilizando la API de IPC System V, que incluye las funciones `semget()` y `semop()`. Estas funciones permiten la creación, manipulación y sincronización de semáforos en aplicaciones concurrentes.

Definición

`semget()` se utiliza para crear o acceder a un conjunto de semáforos. Un conjunto de semáforos es un grupo de semáforos que pueden gestionarse de forma conjunta.

Prototipo

```
int semget(key_t key, int nsems, int semflg);
```

Argumentos

1. `key` (tipo `key_t`)

- Identificador único para el conjunto de semáforos.
- Puede ser generado con `ftok()` o el valor especial `IPC_PRIVATE`.
- Si un conjunto con esta clave ya existe, se accede a él en lugar de crear uno nuevo.

2. `nsems` (tipo `int`)

- Número de semáforos en el conjunto.
- Si el conjunto ya existe, este valor se ignora.
- Si se crea un nuevo conjunto, este número define cuántos semáforos contendrá.

3. **semflg (tipo int)**

- Bandera que indica los permisos y opciones de creación.
- Combina valores como:
 - **IPC_CREAT**: Crea el conjunto de semáforos si no existe.
 - **IPC_EXCL**: Si se usa junto con IPC_CREAT, causa un error si el conjunto ya existe.
 - Permisos en estilo octal, e.g., 0666 para lectura y escritura.

Retorno

- Devuelve un **identificador del conjunto de semáforos** en caso de éxito (entero positivo).
- Devuelve **-1** si hay un error y asigna a errno el código correspondiente.

Errores comunes

- EEXIST: El conjunto ya existe y se usa IPC_CREAT | IPC_EXCL.
- ENOMEM: Memoria insuficiente para crear el conjunto.
- EINVAL: Parámetros inválidos, e.g., nsems menor o igual a 0.

2. **Función semop()**

Definición

semop() realiza operaciones sobre los semáforos de un conjunto. Estas operaciones pueden incluir incrementar, decrementar o esperar en un semáforo.

Prototipo

```
int semop(int semid, struct sembuf *sops, size_t nsops);
```

Argumentos

1. **semid (tipo int)**
 - Identificador del conjunto de semáforos (retornado por semget()).
2. **sops (tipo struct sembuf *)**
 - Puntero a un arreglo de estructuras sembuf que define las operaciones a realizar sobre los semáforos.
3. **nsops (tipo size_t)**
 - Número de operaciones especificadas en el arreglo sops.

Estructura sembuf

Definida en <sys/sem.h>:

```
struct sembuf {  
    unsigned short sem_num; // Índice del semáforo en el conjunto.  
    short sem_op;           // Operación a realizar:  
                            // + Incrementar (>0)  
                            // - Decrementar (<0)  
                            // 0 Esperar (bloqueo hasta que el valor sea 0)  
    short sem_flg;         // Bandera para control de la operación:  
                            // IPC_NOWAIT (no bloquea si falla)  
                            // SEM_UNDO (reversa si el proceso termina)  
};
```

Retorno

- Devuelve **0** si tiene éxito.
- Devuelve **-1** si hay un error, y asigna a errno el código correspondiente.

Errores comunes

- EAGAIN: Operación no puede completarse de inmediato y se especificó IPC_NOWAIT.
- EIDRM: El conjunto de semáforos ha sido eliminado.
- EINVAL: Parámetros inválidos (e.g., índice del semáforo fuera de rango).
- EACCES: Permisos insuficientes para realizar la operación.

2. Capture, compile y ejecute el siguiente programa. Observe su funcionamiento y explique.

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdlib.h>
#include <unistd.h>
int main(void)
{
    int i,j;
    int pid;
    int semid;
    key_t llave = 1234;
    int semban = IPC_CREAT | 0666;
    int nsems = 1;
    int nsops;
    struct sembuf *sops = (struct sembuf *) malloc(2*sizeof(struct sembuf));
    printf("Iniciando semaforo...\n");
    if ((semid = semget(llave, nsems, semban)) == -1) {
        perror("semget: error al iniciar semaforo");
        exit(1);
    }
    else
        printf("Semaforo iniciado...\n");
    if ((pid = fork()) < 0) {
        perror("fork: error al crear proceso\n");
        exit(1);
    }
    if (pid == 0) {
        i = 0;
        while (i < 3) {
            nsops = 2;
            sops[0].sem_num = 0;
            sops[0].sem_op = 0;
            sops[0].sem_flg = SEM_UNDO;

            sops[1].sem_num = 0;
            sops[1].sem_op = 1;
            sops[1].sem_flg = SEM_UNDO | IPC_NOWAIT;
            printf("semaforo: hijo llamando a semop(%d, &sops, %d) con:", semid, nsops);
            for (j = 0; j < nsops; j++) {
                printf("\n\t%sops[%d].sem_num = %d, ", j, sops[j].sem_num);
                printf("sem_op = %d, ", sops[j].sem_op);
                printf("sem_flg = %d\n", sops[j].sem_flg);
            }
            if ((j = semop(semid, sops, nsops)) == -1) {
                perror("semop: error en operacion del semaforo\n");
            }
            else {
                printf("\tsemop: regreso de semop() %d\n", j);
                printf("\n\nProceso hijo toma el control del semaforo: %d/3 veces\n", i+1);
                sleep(5);
                nsops = 1;
                sops[0].sem_num = 0;
                sops[0].sem_op = -1;
                sops[0].sem_flg = SEM_UNDO | IPC_NOWAIT;
                if ((j = semop(semid, sops, nsops)) == -1) {
                    perror("semop: error en operacion del semaforo\n");
                }
                else
                    printf("Proceso hijo regresa el control del semaforo: %d/3 veces\n", i+1);
                sleep(5);
                ++i;
            }
        }
    }
    else {
        i = 0;
        while (i < 3) {
            nsops = 2;
            sops[0].sem_num = 0;
            sops[0].sem_op = 0;
            sops[0].sem_flg = SEM_UNDO;
            sops[1].sem_num = 0;
            sops[1].sem_op = 1;
            sops[1].sem_flg = SEM_UNDO | IPC_NOWAIT;
            printf("\nsemop: Padre llamando semop(%d, &sops, %d) con:", semid, nsops);
            for (j = 0; j < nsops; j++) {
                printf("\n\t%sops[%d].sem_num = %d, ", j, sops[j].sem_num);
                printf("sem_op = %d, ", sops[j].sem_op);
                printf("sem_flg = %d\n", sops[j].sem_flg);
            }
            if ((j = semop(semid, sops, nsops)) == -1) {
                perror("semop: error en operacion del semaforo\n");
            }
            else {
                printf("\tsemop: regreso de semop() %d\n", j);
                printf("\n\nProceso padre toma el control del semaforo: %d/3 veces\n", i+1);
                sleep(5);
                nsops = 1;
                sops[0].sem_num = 0;
                sops[0].sem_op = -1;
                sops[0].sem_flg = SEM_UNDO | IPC_NOWAIT;
                if ((j = semop(semid, sops, nsops)) == -1) {
                    perror("semop: error en semop()\n");
                }
                else
                    printf("Proceso padre regresa el control del semaforo: %d/3 veces\n", i+1);
                sleep(5);
                ++i;
            }
        }
    }
}
```

```
if ((j = semop(semid, sops, nsops)) == -1) {
    perror("semop: error en operacion del semaforo\n");
}
else {
    printf("semop: regreso de semop() %d\n", j);
    printf("Proceso padre toma el control del semaforo: %d/3 veces\n", i+1);
    sleep(5);
    nsops = 1;
    sops[0].sem_num = 0;
    sops[0].sem_op = -1;
    sops[0].sem_flg = SEM_UNDO | IPC_NOWAIT;
    if ((j = semop(semid, sops, nsops)) == -1) {
        perror("semop: error en semop()\n");
    }
    else
        printf("Proceso padre regresa el control del semaforo: %d/3 veces\n", i+1);
    sleep(5);
    ++i;
}
}
```

1. Inicio del programa

- Se declara e inicializa el semáforo:

```
if ((semid = semget(llave, nsems, semban)) == -1) { ... }
```

- Si el semáforo ya existe, lo usa; de lo contrario, lo crea.
- Resultado esperado: El semáforo está listo para sincronizar.

Salida esperada:

Iniciando semaforo...

Semaforo iniciado...

2. Creación del proceso hijo

- Con fork() se genera un proceso hijo.
 - Ambos procesos (padre e hijo) comienzan a ejecutar independientemente.
-

Ciclo de sincronización (3 iteraciones)

3. Proceso Hijo

- Primer paso: Espera a que el semáforo esté disponible (valor 0).

```
sops[0].sem_op = 0;
```

Esto significa que el hijo bloquea su ejecución hasta que el valor del semáforo sea 0.

- Segundo paso: Incrementa el semáforo (lo "toma").

```
sops[1].sem_op = 1;
```

Ahora el semáforo tendrá el valor 1, indicando que está ocupado.

- Imprime que ha tomado el control y duerme 5 segundos para simular trabajo:

```
printf("Proceso hijo toma el control del semaforo...\n");
```

```
sleep(5);
```

- Tercer paso: Libera el semáforo (lo "regresa").

```
sops[0].sem_op = -1;
```

Esto decrementa el valor del semáforo a 0, permitiendo que el proceso padre lo tome.

4. Proceso Padre

- El proceso padre realiza exactamente los mismos pasos que el hijo, pero en paralelo.
- Cuando el hijo incrementa el semáforo a 1, el padre queda bloqueado al intentar ejecutar:

```
sops[0].sem_op = 0;
```

Esto asegura que no se accede simultáneamente al recurso.

Sincronización

La sincronización se logra mediante el valor del semáforo:

- 0: Recurso disponible.
- 1: Recurso ocupado.

Ambos procesos alternan el acceso al recurso de manera ordenada.

```
Iniciando semaforo...
Semaforo iniciado...

semop: Padre llamando semop(0, &sops, 2) con:
      sops[0].sem_num = 0, sem_op = 0, sem_flg = 010000
      sops[1].sem_num = 0, sem_op = 1, sem_flg = 014000
semop: regreso de semop() 0
Proceso padre toma el control del semaforo: 1/3 veces
semop: hijo llamando a semop(0, &sops, 2) con:
      sops[0].sem_num = 0, sem_op = 0, sem_flg = 010000
      sops[1].sem_num = 0, sem_op = 1, sem_flg = 014000
Proceso padre regresa el control del semaforo: 1/3 veces
semop: regreso de semop() 0

Proceso hijo toma el control del semaforo: 1/3 veces

semop: Padre llamando semop(0, &sops, 2) con:
      sops[0].sem_num = 0, sem_op = 0, sem_flg = 010000
      sops[1].sem_num = 0, sem_op = 1, sem_flg = 014000
Proceso hijo regresa el control del semaforo: 1/3 veces
```



```
semop: regreso de semop() 0
Proceso padre toma el control del semaforo: 2/3 veces
semop: hijo llamando a semop(0, &sops, 2) con:
    sops[0].sem_num = 0, sem_op = 0, sem_flg = 010000
Proceso padre regresa el control del semaforo: 2/3 veces

    sops[1].sem_num = 0, sem_op = 1, sem_flg = 014000
semop: regreso de semop() 0
```

```
Proceso hijo toma el control del semaforo: 2/3 veces
Proceso hijo regresa el control del semaforo: 2/3 veces
```

```
semop: Padre llamando semop(0, &sops, 2) con:
    sops[0].sem_num = 0, sem_op = 0, sem_flg = 010000

    sops[1].sem_num = 0, sem_op = 1, sem_flg = 014000
semop: regreso de semop() 0
Proceso padre toma el control del semaforo: 3/3 veces
semop: hijo llamando a semop(0, &sops, 2) con:
    sops[0].sem_num = 0, sem_op = 0, sem_flg = 010000

    sops[1].sem_num = 0, sem_op = 1, sem_flg = 014000
Proceso padre regresa el control del semaforo: 3/3 veces
```

```
semop: regreso de semop() 0
```

```
Proceso hijo toma el control del semaforo: 3/3 veces
Proceso hijo regresa el control del semaforo: 3/3 veces
```

3. A través de la ayuda del sitio MSDN, investigue el funcionamiento de las funciones: CreateSemaphore(), OpenSemaphore(), y ReleaseSemaphore. Explique los argumentos, retorno de las funciones y las estructuras relacionadas con dichas funciones.

1. CreateSemaphore()

Descripción

CreateSemaphore() crea o abre un objeto de semáforo que puede ser utilizado para la sincronización entre procesos o hilos en un sistema Windows. El semáforo mantiene un contador que se incrementa o decrementa dependiendo de las operaciones realizadas.

Prototipo

```
HANDLE CreateSemaphore(  
    LPSECURITY_ATTRIBUTES lpSemaphoreAttributes,  
    LONG lInitialCount,  
    LONG lMaximumCount,  
    LPCWSTR lpName  
);
```

Argumentos

1. lpSemaphoreAttributes

- Tipo: LPSECURITY_ATTRIBUTES
- Una estructura opcional que especifica la seguridad del objeto.
- Si es NULL, el objeto tendrá un descriptor de seguridad predeterminado.

2. lInitialCount

- Tipo: LONG
- Especifica el contador inicial del semáforo. Debe estar entre 0 y lMaximumCount.

3. lMaximumCount

- Tipo: LONG
- Define el valor máximo del contador del semáforo. Este valor debe ser mayor o igual a 1.

4. **lpName**

- Tipo: LPCWSTR
- Especifica un nombre único para el semáforo. Si es NULL, el semáforo no tendrá nombre y será local al proceso.

Valor de retorno

- **Éxito:**
Devuelve un HANDLE que representa el objeto de semáforo creado o abierto.
- **Fallo:**
Devuelve NULL. Para obtener más información sobre el error, se usa GetLastError().

Estructura Relacionada

- **SECURITY_ATTRIBUTES**
 - Controla el acceso al semáforo y si puede heredarse por procesos secundarios.

2. OpenSemaphore()

Descripción

OpenSemaphore() abre un semáforo existente por su nombre, permitiendo que procesos adicionales lo utilicen.

Prototipo

```
HANDLE OpenSemaphore(  
    DWORD dwDesiredAccess,  
    BOOL bInheritHandle,  
    LPCWSTR lpName  
);
```

Argumentos

1. dwDesiredAccess

- Tipo: DWORD
- Especifica los derechos de acceso requeridos para el semáforo. Algunos valores comunes son:
 - SEMAPHORE_ALL_ACCESS: Todos los derechos posibles.
 - SYNCHRONIZE: Permite usar el semáforo para sincronización.

2. bInheritHandle

- Tipo: BOOL
- Especifica si los procesos secundarios pueden heredar el identificador.
 - TRUE: Puede heredarse.
 - FALSE: No puede heredarse.

3. lpName

- Tipo: LPCWSTR
- Nombre del semáforo que se desea abrir. Este debe coincidir con el nombre usado en CreateSemaphore().

Valor de retorno

- **Éxito:**
Devuelve un HANDLE al objeto del semáforo existente.
- **Fallo:**
Devuelve NULL. Se puede obtener información adicional usando GetLastError().

3. ReleaseSemaphore()

Descripción

ReleaseSemaphore() incrementa el contador del semáforo. Si el valor del contador era 0, permite que un hilo en espera continúe su ejecución.

Prototipo

```
BOOL ReleaseSemaphore(  
    HANDLE hSemaphore,  
    LONG IReleaseCount,  
    LPLONG lpPreviousCount  
);
```

Argumentos

1. hSemaphore

- Tipo: HANDLE
- Identificador del semáforo que se desea liberar. Este identificador debe haber sido creado con `CreateSemaphore()` o abierto con `OpenSemaphore()`.

2. IReleaseCount

- Tipo: LONG
- Especifica cuánto se incrementará el contador del semáforo.
 - Si este valor hace que el contador exceda el máximo (`IMaximumCount`), la función falla.

3. lpPreviousCount

- Tipo: LPLONG
 - Dirección de una variable que recibirá el valor del contador del semáforo antes de la operación.
 - Si es NULL, este valor no se devuelve.
-

Valor de retorno

- **Éxito:**
Devuelve TRUE.
- **Fallo:**
Devuelve FALSE. Para obtener detalles, se usa `GetLastError()`.

4. Capture, compile y ejecute los siguientes programas. Observe su funcionamiento. Ejecute de la siguiente forma:
C:\>nombre_programa_padre nombre_programa_hijo

```
#include <windows.h>    /*Programa padre*/
#include <stdio.h>

int main(int argc, char *argv[])
{
    STARTUPINFO si;      // Estructura de información inicial para Windows
    PROCESS_INFORMATION pi; // Estructura de información del administrador de procesos
    HANDLE hSemaforo;
    int i = 1;

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    if (argc != 2) {
        printf("Usar: %s Nombre_programa_hijo\n", argv[0]);
        return;
    }

    // Creación del semáforo
    if ((hSemaforo = CreateSemaphore(NULL, 1, 1, "Semaforo")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }

    // Creación del proceso hijo
    if (!CreateProcess(NULL, argv[1], NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
        printf("Falla al invocar CreateProcess: %d\n", GetLastError());
        return -1;
    }

    // Sección crítica
    while (i < 4) {
        // Prueba del semáforo
        WaitForSingleObject(hSemaforo, INFINITE);

        // Sección crítica
        printf("Soy el padre entrando %i de 3 veces al semaforo\n", i);
        Sleep(5000);

        // Liberación del semáforo
        if (!ReleaseSemaphore(hSemaforo, 1, NULL)) {
            printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
        }
        printf("Soy el padre liberando %i de 3 veces al semaforo\n", i);
        Sleep(5000);

        i++;
    }

    // Terminación controlada del proceso e hilo asociado de ejecución
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

```

#include <windows.h>    /*Programa hijo*/
#include <stdio.h>

int main() {
    HANDLE hSemaforo;
    int i = 1;

    // Apertura del semáforo
    if ((hSemaforo = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "Semaforo")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }

    while (i < 4) {
        // Prueba del semáforo
        WaitForSingleObject(hSemaforo, INFINITE);

        // Sección crítica
        printf("Soy el hijo entrando %i de 3 veces al semaforo\n", i);
        Sleep(5000);

        // Liberación del semáforo
        if (!ReleaseSemaphore(hSemaforo, 1, NULL)) {
            printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
        }
        printf("Soy el hijo liberando %i de 3 veces al semaforo\n", i);
        Sleep(5000);

        i++;
    }
}

```

1. Programa Padre

Este programa crea un semáforo y un proceso hijo, y luego entra en una sección crítica tres veces, bloqueando y liberando el semáforo para asegurar que solo un proceso (el padre o el hijo) pueda ejecutar la sección crítica en un momento dado.

1. Creación del semáforo:

- El padre crea un semáforo llamado "Semaforo" con el valor inicial de 1 (esto significa que el semáforo está disponible para que el primer proceso que lo pida pueda acceder).
- La función `CreateSemaphore` se utiliza para crear el semáforo. Si la creación falla, se imprime un mensaje de error y el programa termina.

2. Creación del proceso hijo:

- El padre crea un proceso hijo con la función `CreateProcess`. Este proceso hijo será el programa que se pasa como argumento al ejecutar el programa padre.
- Si el proceso hijo no se crea correctamente, el padre imprime un mensaje de error y termina la ejecución.

3. Sección crítica (con sincronización de semáforo):

- El padre entra en un bucle donde realiza tres iteraciones. En cada iteración, el padre sigue estos pasos:
 - Esperar el semáforo: Llama a `WaitForSingleObject(hSemaforo, INFINITE)`. Esto bloquea la ejecución del padre hasta que el semáforo esté disponible (es decir, hasta que no haya otro proceso usando la sección crítica).
 - Acceso a la sección crítica: Cuando el semáforo está disponible, el padre puede imprimir un mensaje y simula trabajo con `Sleep(5000)`, lo que hace que el programa se "detenga" durante 5 segundos.
 - Liberar el semáforo: Luego, el padre llama a `ReleaseSemaphore(hSemaforo, 1, NULL)` para liberar el semáforo, permitiendo que el siguiente proceso que lo solicite (el hijo, por ejemplo) pueda acceder a la sección crítica.
 - Espera después de liberar el semáforo: El padre también duerme durante 5 segundos después de liberar el semáforo para simular trabajo o espera.

4. Cierre de handles:

- Una vez que el padre termina su trabajo, cierra los handles del proceso hijo (`pi.hProcess` y `pi.hThread`) para limpiar los recursos utilizados por el proceso hijo.

2. Programa Hijo

Este programa es muy similar al programa padre, pero en lugar de crear el semáforo, abre el semáforo creado por el padre. Luego entra en una sección crítica donde accede y libera el semáforo tres veces, al igual que el padre.

1. Apertura del semáforo:

- El hijo abre el semáforo con la función `OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "Semaforo")`. La diferencia es que el semáforo ya fue creado por el

padre, y el hijo simplemente lo abre para poder usarlo. Si no puede abrir el semáforo, imprime un mensaje de error y termina el programa.

2. Sección crítica (con sincronización de semáforo):

- Al igual que el padre, el hijo entra en un bucle de tres iteraciones. En cada iteración:
 - Esperar el semáforo: Llama a WaitForSingleObject(hSemaforo, INFINITE) para esperar a que el semáforo esté disponible y poder entrar a la sección crítica.
 - Acceso a la sección crítica: Cuando tiene acceso al semáforo, imprime un mensaje indicando que está entrando a la sección crítica y simula trabajo con Sleep(5000).
 - Liberar el semáforo: Después de terminar su trabajo, libera el semáforo con ReleaseSemaphore(hSemaforo, 1, NULL), lo que permitirá que el padre o el hijo (el siguiente proceso) puedan acceder a la sección crítica.
 - Espera después de liberar el semáforo: Al igual que el padre, el hijo también duerme durante 5 segundos después de liberar el semáforo para simular alguna tarea adicional.

3. Fin del programa:

- El hijo termina de ejecutar su trabajo después de tres iteraciones.

```
Soy el padre entrando 1 de 3 veces al semaforo
Soy el hijo entrando 1 de 3 veces al semaforo
Soy el padre liberando 1 de 3 veces al semaforo
Soy el padre entrando 2 de 3 veces al semaforo
Soy el hijo liberando 1 de 3 veces al semaforo
Soy el padre liberando 2 de 3 veces al semaforo
Soy el hijo entrando 2 de 3 veces al semaforo
Soy el hijo liberando 2 de 3 veces al semaforo
Soy el padre entrando 3 de 3 veces al semaforo
Soy el padre liberando 3 de 3 veces al semaforo
Soy el hijo entrando 3 de 3 veces al semaforo
Soy el hijo liberando 3 de 3 veces al semaforo
```

5. Programe la misma aplicación del punto 8 de la práctica 7 (tanto para Linux como para Windows), utilizando como máximo tres regiones de memoria compartida para almacenar todas las matrices requeridas por la aplicación. Utilice como mecanismo de sincronización los semáforos revisados en esta práctica tanto para la escritura y como para la lectura de las memorias compartidas. Úselos en los lugares donde haya necesidad de sincronizar el acceso a memoria compartida.

LINUX:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/shm.h>
#include <time.h>
#include <fcntl.h>
#define TAMANIO 10
void llenarMatriz(int matriz[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            matriz[fila][columna] = rand() % 101;
        }
    }
}
void guardarMatriz(double matriz[TAMANIO][TAMANIO], int archivo) {
    char buffer[256];
    for (int i = 0; i < TAMANIO; i++) {
        int len = 0;
        for (int j = 0; j < TAMANIO; j++) {
            len += sprintf(buffer + len, "%lf\t", matriz[i][j]);
        }
        buffer[len++] = '\n';
        write(archivo, buffer, len);
    }
    write(archivo, "\n", 1);
}
void imprimirMatrizEnteros(int matriz[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            printf("%d\t", matriz[fila][columna]);
        }
        printf("\n");
    }
}
```

```

void imprimirMatrizReales(double matriz[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            printf("%lf ", matriz[fila][columna]);
        }
        printf("\n");
    }
}

void multiplicarMatrices(int matrizA[TAMANIO][TAMANIO], int matrizB[TAMANIO][TAMANIO], int resultado[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            resultado[fila][columna] = 0;
            for (int k = 0; k < TAMANIO; k++){
                resultado[fila][columna] += matrizA[fila][k] * matrizB[k][columna];
            }
        }
    }
}

void sumarMatrices(int matrizA[TAMANIO][TAMANIO], int matrizB[TAMANIO][TAMANIO], int resultado[TAMANIO][TAMANIO]){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            resultado[fila][columna] = matrizA[fila][columna] + matrizB[fila][columna];
        }
    }
}

void calcularCofactor(int matriz[TAMANIO][TAMANIO], int temporal[TAMANIO][TAMANIO], int filaOmitida, int columnaOmitida, int tamActual){
    int filaTemp = 0, columnaTemp = 0;
    for (int fila = 0; fila < tamActual; fila++){
        for (int columna = 0; columna < tamActual; columna++){
            if (fila != filaOmitida && columna != columnaOmitida){
                temporal[filaTemp][columnaTemp++] = matriz[fila][columna];
                if (columnaTemp == tamActual - 1){
                    columnaTemp = 0;
                    filaTemp++;
                }
            }
        }
    }
}

int calcularDeterminante(int matriz[TAMANIO][TAMANIO], int tamActual){
    int determinante = 0;
    if (tamActual == 1)
        return matriz[0][0];
    int cofactor[TAMANIO][TAMANIO];
    int signo = 1;
    for (int columna = 0; columna < tamActual; columna++){
        calcularCofactor(matriz, cofactor, 0, columna, tamActual);
        determinante += signo * matriz[0][columna] * calcularDeterminante(cofactor, tamActual - 1);
        signo = -signo;
    }
    return determinante;
}

void calcularAdjunta(int matriz[TAMANIO][TAMANIO], double adjunta[TAMANIO][TAMANIO], int tamActual){
    if (tamActual == 1){
        adjunta[0][0] = 1;
        return;
    }
    int cofactor[TAMANIO][TAMANIO];
    int signo;
    for (int fila = 0; fila < tamActual; fila++){
        for (int columna = 0; columna < tamActual; columna++){
            calcularCofactor(matriz, cofactor, fila, columna, tamActual);
            signo = ((fila + columna) % 2 == 0) ? 1 : -1;
            adjunta[columna][fila] = signo * calcularDeterminante(cofactor, tamActual - 1);
        }
    }
}

void calcularInversa(int matriz[TAMANIO][TAMANIO], double inversa[TAMANIO][TAMANIO]){
    double determinante = (double)calcularDeterminante(matriz, TAMANIO);
    if (determinante == 0){
        printf("La matriz es singular, no tiene inversa.\n");
        return;
    }
    double adjunta[TAMANIO][TAMANIO];
    calcularAdjunta(matriz, adjunta, TAMANIO);
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            inversa[fila][columna] = adjunta[fila][columna] / determinante;
        }
    }
}

```

```

void escribirMemoria(int (*matriz)[TAMANIO], int *shm){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            *shm = matriz[fila][columna];
            shm++;
        }
    }
}

void escribir2Memoria(int (*matrizA)[TAMANIO], int (*matrizB)[TAMANIO], int (*shm)[TAMANIO]){
    escribirMemoria(matrizA, (int *)shm);
    escribirMemoria(matrizB, (int *)shm + (TAMANIO * TAMANIO));
}

void leerMemoria(int (*matriz)[TAMANIO], int *shm){
    for (int fila = 0; fila < TAMANIO; fila++){
        for (int columna = 0; columna < TAMANIO; columna++){
            matriz[fila][columna] = *shm;
            shm++;
        }
    }
}

void leer2Memoria(int (*matrizA)[TAMANIO], int (*matrizB)[TAMANIO], int (*shm)[TAMANIO]){
    leerMemoria(matrizA, (int *)shm);
    leerMemoria(matrizB, (int *)shm + (TAMANIO * TAMANIO));
}

void semaforo(int semid){
    struct sembuf sops[2];
    sops[0].sem_num = 0;
    sops[0].sem_op = 0;
    sops[0].sem_flg = SEM_UNDO;
    sops[1].sem_num = 0;
    sops[1].sem_op = 1;
    sops[1].sem_flg = SEM_UNDO | IPC_NOWAIT;
    if (semop(semid, sops, 2) == -1){
        perror("Error en operacion del semaforo");
        exit(1);
    }
}

```

```

void liberar(int semid){
    struct sembuf sop;
    sop.sem_num = 0;
    sop.sem_op = -1;
    sop.sem_flg = SEM_UNDO | IPC_NOWAIT;
    if (semop(semid, &sop, 1) == -1){
        perror("Error en operacion del semaforo");
        exit(1);
    }
}

int main(){
    srand(time(NULL));
    int semid1, semid2, semid3;
    int matrizA[TAMANIO][TAMANIO], matrizB[TAMANIO][TAMANIO];
    int matrizMultiplicacion[TAMANIO][TAMANIO], matrizSuma[TAMANIO][TAMANIO];
    key_t llaveA = 1000, llaveB = 1001, llaveR = 1002, llaveSemId1 = 1003, llaveSemId2 = 1004, llaveSemId3 = 1005;
    int shmIdA = shmget(llaveA, sizeof(int)*TAMANIO*TAMANIO*2, IPC_CREAT | 0666);
    int shmIdB = shmget(llaveB, sizeof(int)*TAMANIO*TAMANIO*2, IPC_CREAT | 0666);
    int shmIdR = shmget(llaveR, sizeof(int)*TAMANIO*TAMANIO*2, IPC_CREAT | 0666);
    int(*shmA)[TAMANIO] = shmat(shmIdA, NULL, 0);
    int(*shmB)[TAMANIO] = shmat(shmIdB, NULL, 0);
    int(*shmR)[TAMANIO] = shmat(shmIdR, NULL, 0);
    if (shmIdA < 0) {
        perror("Error al obtener memoria compartida");
        exit(1);
    }
    if (shmA == (int (*)[TAMANIO]) -1) {
        perror("Error al enlazar la memoria compartida para la matriz uno y dos: shmat");
        exit(1);
    }
    if (shmIdB < 0) {
        perror("Error al obtener memoria compartida");
        exit(1);
    }
    if (shmB == (int (*)[TAMANIO]) -1) {
        perror("Error al enlazar la memoria compartida para la matriz uno y dos: shmat");
        exit(1);
    }
}

```

```

if (shmIdR < 0) {
    perror("Error al obtener memoria compartida");
    exit(1);
}
if (shmR == (int (*)(TAMANIO)) -1) {
    perror("Error al enlazar la memoria compartida para la matriz uno y dos: shmat");
    exit(1);
}
if ((semid1 = semget(llaveSemId1, 1, IPC_CREAT | 0666)) == -1){
    perror("error al crear semaforo de comunicacion entre padre e hijo");
    exit(1);
}
if ((semid2 = semget(llaveSemId2, 1, IPC_CREAT | 0666)) == -1){
    perror("error al crear semaforo de comunicacion entre hijo y nieto");
    exit(1);
}
if ((semid3 = semget(llaveSemId3, 1, IPC_CREAT | 0666)) == -1){
    perror("error al crear semaforo resultados");
    exit(1);
}
if (fork() == 0){
    semaforo(semid1);
    leer2Memoria(matrizA,matrizB,shmA);
    multiplicarMatrices(matrizA, matrizB, matrizMultiplicacion);
    escribirMemoria(matrizMultiplicacion,(int *)shmR);
    liberar(semid1);
    semaforo(semid2);
    escribir2Memoria(matrizA,matrizB,shmB);
    liberar(semid2);
    if (fork() == 0){
        semaforo(semid2);
        leer2Memoria(matrizA,matrizB,shmB);
        sumarMatrices(matrizA, matrizB, matrizSuma);
        escribirMemoria(matrizSuma,(int*)shmR+ (TAMANIO*TAMANIO));
        liberar(semid2);
        liberar(semid3);
        exit(0);
    }
    exit(0);
}
}

```

```

else{
    semaforo(semid1);
    llenarMatriz(matrizA);
    llenarMatriz(matrizB);
    printf("Matriz A:\n");
    imprimirMatrizEnteros(matrizA);
    printf("\nMatriz B:\n");
    imprimirMatrizEnteros(matrizB);
    escribir2Memoria(matrizA,matrizB,shmA);
    liberar(semid1);
    semaforo(semid3);
    semaforo(semid3);
    leer2Memoria(matrizMultiplicacion,matrizSuma,shmR);
    printf("\nResultado de la multiplicación:\n");
    imprimirMatrizEnteros(matrizMultiplicacion);
    printf("\nResultado de la suma:\n");
    imprimirMatrizEnteros(matrizSuma);
    double inversa_multi[TAMANIO][TAMANIO];
    double inversa_sum[TAMANIO][TAMANIO];
    calcularInversa(matrizMultiplicacion, inversa_multi);
    calcularInversa(matrizSuma, inversa_sum);
    printf("\nInversa de la multiplicación:\n");
    imprimirMatrizReales(inversa_multi);
    printf("\nInversa de la suma:\n");
    imprimirMatrizReales(inversa_sum);
    int archivo = open("inversaMul.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (archivo < 0) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    write(archivo, "Inversa de la multiplicacion:\n", 30);
    guardarMatriz(inversa_multi, archivo);
    close(archivo);
    int archivo2 = open("inversaSum.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (archivo2 < 0) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    write(archivo2, "Inversa de la suma:\n", 21);
    guardarMatriz(inversa_sum, archivo2);

```

```

        close(archivo2);
        liberar(semid3);
    }
    shmdt(shmA);
    shmdt(shmB);
    shmdt(shmR);
    semctl(semid1, 0, IPC_RMID);
    semctl(semid2, 0, IPC_RMID);
    semctl(semid3, 0, IPC_RMID);
    return 0;
}

```

```
said@said-VirtualBox:~/Descargas$ ./sem
```

```
Matriz A:
```

88	50	81	84	58	83	84	71	83	90
1	47	11	100	27	34	19	71	93	99
48	50	89	45	6	33	35	47	1	80
61	90	97	41	39	20	23	23	57	5
79	59	19	90	24	46	91	43	83	49
41	30	99	97	75	71	29	9	18	98
55	79	87	17	86	25	37	8	14	95
80	59	19	99	15	43	10	5	53	93
20	94	22	18	56	63	90	86	38	74
49	94	18	35	77	3	26	14	11	7

```
Matriz B:
```

75	91	66	94	56	47	36	66	52	89
25	72	49	13	57	4	77	12	56	14
86	4	74	70	5	51	73	99	31	84
5	5	40	37	99	62	85	0	95	2
56	19	41	71	32	64	75	75	42	98
56	27	68	29	63	74	46	35	38	43
85	9	48	91	12	12	53	63	13	47
32	35	66	73	5	64	2	46	39	44
43	95	37	78	90	100	51	2	34	55
45	18	64	60	8	42	72	27	5	51

```
Resultado de la multiplicación:
```

40163	28441	43183	48926	33137	41161	43546	32836	30590	40984
18453	18723	25977	28184	25441	29470	29567	12256	21624	20160
23035	13049	26433	25934	14094	19499	25628	20713	16558	22393
24043	20422	25264	27078	21159	22350	28069	21090	20671	25084
28289	24752	30636	36725	29627	29640	33010	20413	25234	27196
28937	14164	32209	32727	23774	30004	37678	25900	23774	30729
28161	17088	28747	30490	16532	22092	32990	25808	17902	30552
20326	20519	25799	26581	26587	24891	30267	13976	22314	21407
27862	20293	30992	32983	19653	25292	30400	22731	19628	27055
15674	14913	16960	18603	15738	13519	21675	14517	16336	17734

```
Resultado de la suma:
```

163	141	147	178	114	130	120	137	135	179
26	119	60	113	84	38	96	83	149	113
134	54	163	115	11	84	108	146	32	164
66	95	137	78	138	82	108	23	152	7
135	78	60	161	56	110	166	118	125	147
97	57	167	126	138	145	75	44	56	141
140	88	135	108	98	37	90	71	27	142
112	94	85	172	20	107	12	51	92	137
63	189	59	96	146	163	141	88	72	129
94	112	82	95	85	45	98	41	16	58

Inversa de la multiplicación:

```
-1.137109 0.521659 0.796922 -0.631163 0.663979 1.164462 1.575796 -1.260122
-0.623419 0.733849
1.546289 0.600021 -0.918091 -0.686141 0.600043 0.144699 0.837123 -0.215070
-0.462787 -0.825002
-0.361967 0.488567 -1.325453 1.086252 0.387212 1.505343 0.492399 1.064612
-1.429001 -0.059017
-0.651579 -0.526767 -1.332178 -1.233149 0.586363 -1.309870 -0.157034 -1.574
548 -0.897091 0.993677
1.052108 -1.364524 -0.804680 -0.833645 0.948418 0.630088 0.955353 0.504784
-0.963601 0.972751
0.004831 0.157838 -0.266900 -0.981336 0.000921 0.208364 1.576150 -0.088188
0.836803 -0.390194
0.951028 -1.203568 0.664585 1.192915 -0.797201 -1.252315 -0.364410 -0.67920
7 -0.110119 -0.005632
-0.527036 -1.432616 -0.949574 1.566642 0.637779 0.697291 -1.550686 -0.96894
3 0.146003 -0.226909
0.018516 -0.838942 0.500954 -1.379181 -1.213632 -0.095018 1.484106 0.445656
-1.469028 -0.313345
-1.489036 -0.648058 -0.309855 1.257285 0.569296 -1.499199 -0.797278 0.84892
1 -0.951932 0.978167
```

Inversa de la suma:

```
-1.302312 -3.192894 -0.943157 -0.521759 -3.384494 -2.588910 -2.778888 -1.52
5189 2.912687 1.646752
-1.637849 -3.910607 -0.994377 3.702427 0.236858 1.081249 1.279410 -1.904853
-0.106510 -0.364392
1.550319 -2.739466 -2.825770 -1.700462 0.022975 -1.289006 3.704391 -3.34666
4 3.148584 3.115838
0.098878 -3.909070 -1.127223 -0.445414 -2.362411 -0.415021 -0.577392 -0.862
045 -0.137595 3.879043
-3.572668 -0.003215 -1.266179 1.877504 3.596376 -2.062139 -2.032427 0.51129
4 3.330346 2.998038
2.440757 0.098731 1.494920 3.139046 3.910480 2.637816 1.397702 2.403341 0.
618209 0.044336
-0.777999 -2.623753 1.969638 -1.279299 -3.241686 1.939538 0.279543 -3.60014
2 -0.752464 -1.387727
0.561340 -3.530274 0.143264 -2.821600 1.450598 -0.496428 1.154361 -3.195302
1.211774 -0.765651
0.025575 1.195327 -1.749672 3.704479 0.649900 2.535550 -3.899465 -2.150137
-1.448535 -1.108863
1.904349 -1.673911 2.986901 -0.090025 1.455943 -1.552151 0.503744 -0.431464
3.192665 1.185871
```



inversaSum.
txt



inversaMul.t
xt

Inversa de la suma:

```
\00-1.302312    -3.192894    -0.943157    -0.521759    -3.
...
384494  -2.588910    -2.778888    -1.525189    2.912687
1.646752
-1.637849    -3.910607    -0.994377    3.702427
0.236858    1.081249
1.279410    -1.904853    -0.106510    -0.364392
1.550319    -2.739466    -2.825770    -1.700462
0.022975    -1.289006    3.704391    -3.346664
3.148584    3.115838
0.098878    -3.909070    -1.127223    -0.445414    -2.
362411  -0.415021    -0.577392    -0.862045    -0.137595
3.879043
-3.572668    -0.003215    -1.266179    1.877504
```

Inversa de la multiplicacion:

```
...
-1.137109    0.521659    0.796922    -0.631163
0.663979    1.164462
1.575796    -1.260122    -0.623419    0.733849
1.546289    0.600021    -0.918091    -0.686141
0.600043    0.144699
0.837123    -0.215070    -0.462787    -0.825002
-0.361967    0.488567    -1.325453    1.086252
0.387212    1.505343    0.492399
1.064612    -1.429001    -0.059017
-0.651579    -0.526767    -1.332178    -1.233149
0.586363    -1.309870    -0.157034    -1.574548    -0.
897091  0.993677
1.052108    -1.364524    -0.804680    -0.833645
0.948418    0.630088    0.955353
0.504784    -0.963601    0.972751
```

WINDOWS

Padre.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void LlenarMatriz(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = rand() % 101;
        }
    }
}
void ImprimirMatriz(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
void impInversaArchivo(double matriz[N][N], FILE *archivo){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            fprintf(archivo, "%lf\t", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fprintf(archivo, "\n");
}
void ImprimirInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++)
            printf("%lf ", matriz[i][j]);
        printf("\n");
    }
}
```

```

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++){
        for (int col = 0; col < n; col++){
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1){
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n){
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++){
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1){
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

```

```

void InversaMatriz(int matriz[N][N], double inversa[N][N]){
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0){
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

HANDLE CrearMemoriaCompartidaDoble(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, (2 * N * N * sizeof(int)), nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, (2 * N * N * sizeof(int)))) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void EscribirMemoriaCompartida2(int matrizA[N][N], int matrizB[N][N], int* punteroDatos){
    EscribirMemoriaCompartida(matrizA, punteroDatos);
    EscribirMemoriaCompartida(matrizB, punteroDatos + (N * N));
}

```

```

int* AbrirMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE, nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile(hMemCom, FILE_MAP_ALL_ACCESS, 0, 0, (2 * N * N * sizeof(int)))) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

void LeerMemoriaCompartida2(int matrizA[N][N], int matrizB[N][N], int* punteroDatos){
    LeerMemoriaCompartida(matrizA, punteroDatos);
    LeerMemoriaCompartida(matrizB, punteroDatos + (N * N));
}

int main(int argc, char* argv[]){
    srand(time(NULL));
    int matrizA[N][N];
    int matrizB[N][N];
    LlenarMatriz(matrizA);
    LlenarMatriz(matrizB);
    printf("Matriz A:\n");
    ImprimirMatriz(matrizA);
    printf("\nMatriz B:\n");
    ImprimirMatriz(matrizB);
    int* apDatos;
    HANDLE MemCom1 = CrearMemoriaCompartidaDoble("MemCom1", &apDatos);
    EscribirMemoriaCompartida2(matrizA, matrizB, apDatos);
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
}

```

```

ZeroMemory(&pi, sizeof(pi));
if (!CreateProcess(NULL, argv[1], NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)){
    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
    return 1;
}
HANDLE semaforomat;
if ((semaforomat = CreateSemaphore(NULL, 0, 1, "semaforomat")) == NULL) {
    printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
    return -1;
}
WaitForSingleObject(semaforomat, INFINITE);
UnmapViewOfFile(apDatos);
CloseHandle(MemCom1);
HANDLE semaforores;
if ((semaforores = CreateSemaphore(NULL, 0, 1, "semaforores")) == NULL) {
    printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
    return -1;
}
WaitForSingleObject(semaforores, INFINITE);
int Resultado_multi[N][N];
int Resultado_sum[N][N];
HANDLE MemCom2;
int* apDatos2;
apDatos2 = AbrirMemoriaCompartida("MemCom2", MemCom2);
LeerMemoriaCompartida2(Resultado_multi, Resultado_sum, apDatos2);
UnmapViewOfFile(apDatos2);
CloseHandle(MemCom2);
printf("\nResultado de la suma:\n\n");
ImprimirMatriz(Resultado_sum);
printf("\nResultado de la multiplicacion:\n\n");
ImprimirMatriz(Resultado_multi);
HANDLE semaforores2;
if ((semaforores2 = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semaforores2")) == NULL) {
    printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
    return -1;
}
if (!ReleaseSemaphore(semaforores2, 1, NULL)) {
    printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
}
}

```

```

double Inversa_multi[N][N];
double Inversa_sum[N][N];
InversaMatriz(Resultado_multi, Inversa_multi);
InversaMatriz(Resultado_sum, Inversa_sum);
printf("\nInversa del resultado de la multiplicacion:\n\n");
ImprimirInversa(Inversa_multi);
FILE *archivo;
archivo = fopen("inversaMul.txt", "w");
if (!archivo) {
    printf("Error abriendo archivo para multiplicación.\n");
    return 1;
}
impInversaArchivo(Inversa_multi, archivo);
fclose(archivo);
printf("\nInversa del resultado de la suma:\n\n");
ImprimirInversa(Inversa_sum);
archivo = fopen("inversaSum.txt", "w");
if (!archivo) {
    printf("Error abriendo archivo para suma.\n");
    return 1;
}
impInversaArchivo(Inversa_sum, archivo);
fclose(archivo);
WaitForSingleObject(pi.hProcess, INFINITE);
CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);
return 0;
}

```

Hijo.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void MultiplicarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++){
                resultado[i][j] += matrizuno[i][k] * matrizdos[k][j];
            }
        }
    }
}

int* AbrirMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE, nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile(hMemCom, FILE_MAP_ALL_ACCESS, 0, 0, (2 * N * N * sizeof(int)))) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}
```

```

void LeerMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    LeerMemoriaCompartida(matrizuno, punteroDatos);
    LeerMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}

HANDLE CrearMemoriaCompartidaDoble(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, (2 * N * N * sizeof(int)), nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, (2 * N * N * sizeof(int)))) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

void EscribirMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos)
{
    EscribirMemoriaCompartida(matrizuno, punteroDatos);
    EscribirMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE memComMat;
    int* apDatos1;
    apDatos1 = AbrirMemoriaCompartida("MemCom1", memComMat);
    LeerMemoriaCompartida2(Matrizuno, Matrizdos, apDatos1);
    UnmapViewOfFile(apDatos1);
    CloseHandle(memComMat);
    HANDLE semaforo1;
    if ((semaforo1 = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semaforomat")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semaforo1, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
}

```

```

int Resultado[N][N];
MultiplicarMatrices(Matrizuno, Matrizdos, Resultado);
int* apDatos2;
HANDLE memCom2 = CrearMemoriaCompartidaDoble("MemCom2", &apDatos2);
EscribirMemoriaCompartida(Resultado, apDatos2);
int* apDatos;
HANDLE hMemCom = CrearMemoriaCompartidaDoble("MemCom3", &apDatos);
EscribirMemoriaCompartida2(Matrizuno, Matrizdos, apDatos);
STARTUPINFO si;
PROCESS_INFORMATION pi;
ZeroMemory(&si, sizeof(si));
si.cb = sizeof(si);
ZeroMemory(&pi, sizeof(pi));
if (!CreateProcess(NULL, "nieto", NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)){
    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
    return 1;
}

HANDLE semaforo2;
if ((semaforo2 = CreateSemaphore(NULL, 0, 1, "semaforo2")) == NULL) {
    printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
    return -1;
}

WaitForSingleObject(semaforo2, INFINITE);
UnmapViewOfFile(apDatos);
CloseHandle(hMemCom);
WaitForSingleObject(pi.hProcess, INFINITE);
UnmapViewOfFile(apDatos2);
CloseHandle(memCom2);
CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);
return 0;
}

```

Nieto.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (N * N * sizeof(int))
void SumarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = matrizuno[i][j] + matrizdos[i][j];
        }
    }
}

int* AbrirMemoriaCompartida(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile(hMemCom,FILE_MAP_ALL_ACCESS, 0,0,(2 * N * N * sizeof(int)))) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

void LeerMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    LeerMemoriaCompartida(matrizuno, punteroDatos);
    LeerMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}
```



```

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    punteroDatos += N * N;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

int main(void){
    int matrizA[N][N];
    int matrizB[N][N];
    HANDLE memCom1;
    int* apDatos1;
    apDatos1 = AbrirMemoriaCompartida("MemCom3", memCom1);
    LeerMemoriaCompartida2(matrizA, matrizB, apDatos1);
    UnmapViewOfFile(apDatos1);
    CloseHandle(memCom1);
    HANDLE semaforo1;
    if ((semaforo1 = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semaforo2")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semaforo1, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    int Resultado[N][N];
    SumarMatrices(matrizA, matrizB, Resultado);
    HANDLE memCom2;
    int* apDatos2;
    apDatos2 = AbrirMemoriaCompartida("MemCom2", memCom2);
    EscribirMemoriaCompartida(Resultado, apDatos2);
    UnmapViewOfFile(apDatos2);
    CloseHandle(memCom2);
    HANDLE semaforo2;
    if ((semaforo2 = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semaforores")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semaforo2, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
}

```

```

    HANDLE semaforo3;
    if ((semaforo3 = CreateSemaphore(NULL, 0, 1, "semaforores2")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
    WaitForSingleObject(semaforo3, INFINITE);
    return 0;
}

```

C:\Users\vbcsl>padre hijo

Matriz A:

89	58	87	60	6	10	60	90	24	37
91	51	100	73	53	96	1	63	71	41
6	2	43	25	90	40	12	19	34	59
20	43	65	6	53	57	22	55	51	82
90	54	64	99	79	72	54	76	10	38
26	16	99	84	26	26	45	36	3	79
51	69	79	95	38	76	64	80	25	74
14	9	58	24	24	70	85	79	8	37
46	58	8	24	19	70	22	24	84	70
19	18	36	62	13	23	67	12	37	88

Matriz B:

55	94	40	87	57	0	97	58	79	75
82	31	45	76	61	4	55	95	2	65
76	69	18	15	76	90	36	79	25	97
25	83	10	54	20	92	13	99	29	15
73	27	29	51	73	12	93	82	61	47
6	19	77	8	97	98	3	20	1	82
9	15	8	87	25	7	32	1	83	55
65	54	22	45	62	42	3	95	6	58
4	45	65	40	51	44	28	92	0	31
94	75	31	20	14	95	36	9	84	75

Resultado de la suma:

144	152	127	147	63	10	157	148	103	112
173	82	145	149	114	100	56	158	73	106
82	71	61	40	166	130	48	98	59	156
45	126	75	60	73	149	35	154	80	97
163	81	93	150	152	84	147	158	71	85
32	35	176	92	123	124	48	56	4	161
60	84	87	182	63	83	96	81	108	129
79	63	80	69	86	112	88	174	14	95
50	103	73	64	70	114	50	116	84	101
113	93	67	82	27	118	103	21	121	163

Resultado de la multiplicacion:

28225	31114	14447	28052	26653	23405	20517	35328	20066	32925
31299	36036	24674	27288	38665	35636	25083	44403	19142	40283
18222	16019	11597	12018	18720	19161	14572	20066	13874	19877
25612	22455	17134	17535	25930	25294	17072	27137	15799	29986
31954	34482	19939	32123	35123	30708	26458	41661	24710	38108
24603	26618	10934	18268	20698	29026	15030	26269	19196	27788
32904	34694	20662	30190	34037	35968	22235	40425	23299	39901
18098	18243	12420	17862	22726	22392	11208	20652	15441	26769
18975	21165	19109	19109	22727	21756	15877	24804	13725	25445
17697	20680	10919	17365	14875	22387	12597	18723	18076	21642

Inversa del resultado de la multiplicacion:

```
0.431152 0.677262 -0.937755 0.225626 -0.284724 -1.090890 1.213814 0.791839 -1.050643 0.993353
-0.866786 -0.271748 -0.654083 -1.118253 -0.891643 0.305881 -0.018450 -0.439893 -0.213441 0.886183
-0.742402 -1.132847 0.252862 -0.623507 -0.382919 -0.746291 -0.510584 1.198767 -0.608577 0.869721
0.498013 -0.662959 -1.155918 0.703090 0.376510 1.098721 -0.774623 -0.295706 0.505316 1.023359
0.779720 0.828877 -0.161521 0.231477 -0.894253 0.442686 0.551482 0.603242 -0.621375 -0.135057
-0.131215 0.244661 1.157261 -0.761950 0.881317 -0.377749 -0.325606 0.660602 0.919785 -1.201186
0.671099 1.131299 0.807930 -1.131952 0.228950 0.202850 0.769215 -0.190792 0.909822 -0.188184
0.404810 1.067076 -1.131460 0.381137 -0.690106 -0.698588 -0.214803 0.442073 -0.642028 -1.110677
0.559912 -0.503194 -1.078201 -0.266584 0.925983 0.650419 -0.249732 -0.928966 -0.634179 0.203169
0.587175 0.617800 -1.176609 0.161346 0.167555 -0.169567 0.389067 1.072572 0.263431 0.222889
```

Inversa del resultado de la suma:

```
0.591070 -1.298072 -0.507959 -0.231394 1.185101 -1.057631 0.176093 1.336417 -0.284968 -1.095657
-0.849294 1.216161 -0.648548 1.038622 -1.239356 -0.449426 -0.273402 -0.533618 0.440384 -1.141974
-0.032093 -0.813991 1.049439 -1.376963 -0.994980 -0.301677 -0.168888 1.106739 -0.868937 0.246820
-0.203363 -1.227852 0.970440 0.977564 0.323978 1.234137 0.134291 1.049840 -1.265864 0.861273
0.962233 1.348904 0.977057 0.846579 1.161059 0.460649 -0.320558 -0.805998 -0.186132 -1.035277
-0.292678 -0.773063 -1.300966 -0.563589 -0.004144 1.313106 -1.373470 1.093193 -0.879016 0.226019
-0.454888 -0.690277 0.987995 -1.001973 -0.995047 -0.619409 -0.465184 1.340285 1.262050 -0.074741
-1.020818 0.112897 -1.314838 0.122166 1.007006 1.219271 1.280870 -1.317148 0.902591 -0.044922
-1.232035 0.781038 -0.635729 0.173243 -0.581035 0.341020 -0.921461 -0.278477 0.300084 1.279094
1.119935 -1.013256 -0.820326 -0.717028 -0.226159 0.021771 1.253790 0.241561 0.369493 -0.648440
```



inversaM
ul



inversaS
um

0.431152	0.677262	-0.937755	0.225626	-0.284724	-1.090890	1.213814
0.791839	-1.050643	0.993353				
-0.866786	-0.271748	-0.654083	-1.118253	-0.891643	0.305881	-0.018450
-0.439893	-0.213441	0.886183				
-0.742402	-1.132847	0.252862	-0.623507	-0.382919	-0.746291	-0.510584
1.198767	-0.608577	0.869721				
0.498013	-0.662959	-1.155918	0.703090	0.376510	1.098721	-0.774623
-0.295706	0.505316	1.023359				
0.779720	0.828877	-0.161521	0.231477	-0.894253	0.442686	0.551482
0.603242	-0.621375	-0.135057				
-0.131215	0.244661	1.157261	-0.761950	0.881317	-0.377749	-0.325606
0.660602	0.919785	-1.201186				
0.671099	1.131299	0.807930	-1.131952	0.228950	0.202850	0.769215
-0.190792	0.909822	-0.188184				
0.404810	1.067076	-1.131460	0.381137	-0.690106	-0.698588	-0.214803
0.442073	-0.642028	-1.110677				
0.559912	-0.503194	-1.078201	-0.266584	0.925983	0.650419	-0.249732
-0.928966	-0.634179	0.203169				
0.587175	0.617800	-1.176609	0.161346	0.167555	-0.169567	0.389067
1.072572	0.263431	0.222889				

0.591070	-1.298072	-0.507959	-0.231394	1.185101	-1.057631	0.176093
1.336417	-0.284968	-1.095657				
-0.849294	1.216161	-0.648548	1.038622	-1.239356	-0.449426	-0.273402
-0.533618	0.440384	-1.141974				
-0.032093	-0.813991	1.049439	-1.376963	-0.994980	-0.301677	-0.168888
1.106739	-0.868937	0.246820				
-0.203363	-1.227852	0.970440	0.977564	0.323978	1.234137	0.134291
1.049840	-1.265864	0.861273				
0.962233	1.348904	0.977057	0.846579	1.161059	0.460649	-0.320558
-0.805998	-0.186132	-1.035277				
-0.292678	-0.773063	-1.300966	-0.563589	-0.004144	1.313106	-1.373470
1.093193	-0.879016	0.226019				
-0.454888	-0.690277	0.987995	-1.001973	-0.995047	-0.619409	-0.465184
1.340285	1.262050	-0.074741				
-1.020818	0.112897	-1.314838	0.122166	1.007006	1.219271	1.280870
-1.317148	0.902591	-0.044922				
-1.232035	0.781038	-0.635729	0.173243	-0.581035	0.341020	-0.921461
-0.278477	0.300084	1.279094				
1.119935	-1.013256	-0.820326	-0.717028	-0.226159	0.021771	1.253790
0.241561	0.369493	-0.648440				

6. Programe la misma aplicación del punto 9 de la práctica 7 (tanto para Linux como para Windows) en la versión de memoria compartida, pero en esta ocasión sólo se usarán tres regiones de memoria compartidas para almacenar todas las matrices requeridas por la aplicación. Utilice como mecanismo de sincronización los semáforos revisados en esta práctica tanto para la escritura y como para la lectura de las memorias compartidas. Úselos en los lugares donde haya necesidad de sincronizar el acceso a memoria compartida.

Linux:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/shm.h>
#include <time.h>
#define N 10

void imprimirMatriz(double A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%.2lf\t",A[i][j]);
        }
        printf("\n");
    }
}

void imprimirMatrizI(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t",A[i][j]);
        }
        printf("\n");
    }
}

void sumarMatrices(int A[N][N], int B[N][N], int resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}

void restarMatrices(int A[N][N], int B[N][N], int resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] - B[i][j];
        }
    }
}
```

```

void multiplicarMatrices(int A[N][N], int B[N][N], int resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultado[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void transponerMatriz(int matriz[N][N], int resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[j][i] = matriz[i][j];
        }
    }
}

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

```

```

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1) {
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

void InversaMatriz(int matriz[N][N], double inversa[N][N]) {
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

```

```

void escribirMatriz(int (*matriz)[N], int *shm){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            *shm = matriz[i][j];
            shm++;
        }
    }
}

void escribirMatriz2(int (*matrizuno)[N], int (*matrizdos)[N], int (*shm)[N]){
    escribirMatriz(matrizuno, (int *)shm);
    escribirMatriz(matrizdos, (int *)shm + (N * N));
}

void escribirMemoriaD(double (*matriz)[N], double *shm){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++){
            *shm = matriz[i][j];
            shm++;
        }
    }
}

void leerMatriz(int (*matriz)[N], int *shm){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            matriz[i][j] = *shm;
            shm++;
        }
    }
}

void leerMatriz2(int (*matrizuno)[N], int (*matrizdos)[N], int (*shm)[N]){
    leerMatriz(matrizuno, (int *)shm);
    leerMatriz(matrizdos, (int *)shm + (N * N));
}

void leerMemoriaD(double (*matriz)[N], double *shm) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *shm;
            shm++;
        }
    }
}

```

```

void semaforo(int semid){
    struct sembuf sops[2];
    sops[0].sem_num = 0;
    sops[0].sem_op = 0;
    sops[0].sem_flg = SEM_UNDO;
    sops[1].sem_num = 0;
    sops[1].sem_op = 1;
    sops[1].sem_flg = SEM_UNDO | IPC_NOWAIT;
    if (semop(semid, sops, 2) == -1){
        perror("error en operacion del semaforo");
        exit(1);
    }
}

void liberar(int semid){
    struct sembuf sop;
    sop.sem_num = 0;
    sop.sem_op = -1;
    sop.sem_flg = SEM_UNDO | IPC_NOWAIT;
    if (semop(semid, &sop, 1) == -1){
        perror("error en operacion del semaforo");
        exit(1);
    }
}

int main(){
    srand(time(NULL));
    int semid1, semid2, semid3;
    int A[N][N], B[N][N], TA[N][N], TB[N][N], R[N][N];
    double IA[N][N], IB[N][N];
    double resultado[N][N];
    key_t keysem1 = 1001, keysem2 = 1002, keysem3 = 1003, key1 = 1004, key2 = 1005, key3 = 1006;
    if ((semid1 = semget(keysem1, 1, IPC_CREAT | 0666)) == -1){
        perror("semget: error al crear semaforo de comunicacion 1");
        exit(1);
    }
    if ((semid2 = semget(keysem2, 1, IPC_CREAT | 0666)) == -1){
        perror("semget: error al crear semaforo de comunicacion 2");
        exit(1);
    }
}

```



```

if ((semid3 = semget(keysem3, 1, IPC_CREAT | 0666)) == -1){
    perror("semget: error al crear semaforo de comunicacion 3");
    exit(1);
}
int shmid1 = shmget(key1, sizeof(int[N][N]), IPC_CREAT | 0666);
int shmid2 = shmget(key2, sizeof(int[N][N]), IPC_CREAT | 0666);
int shmid3 = shmget(key3, sizeof(int[N][N]), IPC_CREAT | 0666);
int (*shmMat)[N] = shmat(shmid1, NULL, 0);
int (*shmRes)[N] = shmat(shmid2, NULL, 0);
int (*shmResD)[N] = shmat(shmid3, NULL, 0);
if (shmid1 < 0) {
    perror("Error al obtener memoria compartida para la matriz uno y dos: shmget");
    exit(1);
}
if (shmMat == (void *)-1) {
    perror("Error al adjuntar memoria compartida para la matriz uno y dos: shmat");
    exit(1);
}
if (shmid2 < 0) {
    perror("Error al obtener memoria compartida para los resultados enteros: shmget");
    exit(1);
}
if (shmRes == (void *)-1) {
    perror("Error al adjuntar memoria compartida para los resultados enteros: shmat");
    exit(1);
}
if (shmid3 < 0) {
    perror("Error al obtener memoria compartida para los resultados doubles: shmget");
    exit(1);
}
if (shmResD == (void *)-1) {
    perror("Error al adjuntar memoria compartida para los resultados doubles: shmat");
    exit(1);
}
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        A[i][j] = rand() % 101;
        B[i][j] = rand() % 101;
    }
}
}

```

```

printf("\nMatriz A:\n");
imprimirMatrizI(A);
printf("\nMatriz B:\n");
imprimirMatrizI(B);
escribirMatriz2(A,B,shmMat);
semaforo(semid2);
semaforo(semid3);
pid_t pid;
for (int i = 1; i <= 6; i++) {
    pid = fork();
    if (pid == 0) {
        switch (i) {
            case 1:
                semaforo(semid1);
                leerMatriz2(A,B,shmMat);
                sumarMatrices(A, B, R);
                escribirMatriz(R, (int *)shmRes);
                liberar(semid1);
                break;
            case 2:
                semaforo(semid1);
                leerMatriz2(A,B,shmMat);
                restarMatrices(A, B, R);
                escribirMatriz(R, (int *)shmRes+(N*N));
                liberar(semid1);
                break;
            case 3:
                semaforo(semid1);
                leerMatriz2(A,B,shmMat);
                multiplicarMatrices(A, B, R);
                escribirMatriz(R, (int *)shmRes+(2*(N*N)));
                liberar(semid1);
                break;

```

```

            case 4:
                semaforo(semid1);
                leerMatriz2(A, B, shmMat);
                transponerMatriz(A, TA);
                transponerMatriz(B, TB);
                escribirMatriz(TA, (int *)shmRes + (3 * (N * N)));
                escribirMatriz(TB, (int *)shmRes + (4 * (N * N)));
                liberar(semid1);
                break;
            case 5:
                semaforo(semid1);
                leerMatriz2(A, B, shmMat);
                InversaMatriz(A, IA);
                InversaMatriz(B, IB);
                escribirMemoriaD(IA, (double *)shmResD);
                escribirMemoriaD(IB, (double *)shmResD + (N * N));
                liberar(semid1);
                liberar(semid2);
                break;

```

```

        case 6:
            semaforo(semid2);
            printf("\nSuma:\n");
            leerMatriz(R, (int *)shmRes);
            imprimirMatrizI(R);
            printf("\nResta:\n");
            leerMatriz(R, (int *)shmRes + (N * N));
            imprimirMatrizI(R);
            printf("\nMultiplicacion:\n");
            leerMatriz(R, (int *)shmRes + (2 * (N * N)));
            imprimirMatrizI(R);
            printf("\nTraspuesta A:\n");
            leerMatriz(TA, (int *)shmRes + (3 * (N * N)));
            imprimirMatrizI(TA);
            printf("\nTraspuesta B:\n");
            leerMatriz(TB, (int *)shmRes + (4 * (N * N)));
            imprimirMatrizI(TB);
            printf("\nInversa A:\n");
            leerMemoriaD(IA, (double *)shmResD);
            imprimirMatriz(IA);
            printf("\nInversa B:\n");
            leerMemoriaD(IB, (double *)shmResD + (N * N));
            imprimirMatriz(IB);
            liberar(semid2);
            liberar(semid3);
            break;
    }
    exit(0); } }

semaforo(semid3);
shmdt(shmMat);
shmdt(shmRes);
shmdt(shmResD);
semctl(semid1, 0, IPC_RMID);
semctl(semid2, 0, IPC_RMID);
semctl(semid3, 0, IPC_RMID);
return 0;
}

```

Matriz A:

11	84	37	98	11	67	70	13	95	87
85	5	10	35	22	97	57	57	66	80
19	11	45	82	98	54	44	59	89	44
65	10	53	82	87	96	41	62	64	92
10	8	71	75	95	2	92	13	46	62
23	72	97	82	75	53	85	5	96	46
31	81	28	66	56	39	17	75	76	6
62	36	88	33	54	4	94	73	45	28
98	45	10	62	8	2	94	82	12	96
25	16	51	73	93	63	67	71	60	28

Matriz B:

87	23	54	20	100	68	66	93	6	49
9	78	30	24	4	33	46	94	54	78
13	90	91	39	35	82	59	54	61	78
40	22	67	18	28	73	73	86	9	61
45	20	34	25	85	25	13	44	96	100
24	30	100	26	73	86	62	21	46	56
49	22	95	40	94	45	77	81	22	51
60	12	7	49	45	85	85	88	38	0
33	15	86	52	34	70	4	47	81	32
100	86	89	62	67	90	74	77	32	34

Suma:

98	107	91	118	111	135	136	106	101	136
94	83	40	59	26	130	103	151	120	158
32	101	136	121	133	136	103	113	150	122
105	32	120	100	115	169	114	148	73	153
55	28	105	100	180	27	105	57	142	162
47	102	197	108	148	139	147	26	142	102
80	103	123	106	150	84	94	156	98	57
122	48	95	82	99	89	179	161	83	28
131	60	96	114	42	72	98	129	93	128
125	102	140	135	160	153	141	148	92	62

Resta:

-76	61	-17	78	-89	-1	4	-80	89	38
76	-73	-20	11	18	64	11	-37	12	2
6	-79	-46	43	63	-28	-15	5	28	-34
25	-12	-14	64	59	23	-32	-24	55	31
-35	-12	37	50	10	-23	79	-31	-50	-38
-1	42	-3	56	2	-33	23	-16	50	-10
-18	59	-67	26	-38	-6	-60	-6	54	-45
2	24	81	-16	9	-81	9	-15	7	28
65	30	-76	10	-26	-68	90	35	-69	64
-75	-70	-38	11	26	-27	-7	-6	28	-6

Multiplicacion:

24262	25124	42775	21231	27525	38480	31537	39214	24392	30375
28679	17173	37053	19377	34328	37442	30703	33825	19605	22841
24356	17524	35840	19736	30852	35610	27038	34056	27910	29713
32974	23987	44706	23464	40078	45388	36437	41612	29055	34444
22194	19056	34001	17491	28789	28833	25012	31720	22938	29111
224070	27095	46033	22189	33084	39423	31558	40989	31627	38750
18327	16223	26088	15937	21818	29649	23820	32979	23604	25334
23979	20107	32686	18997	30757	32828	29994	37509	22732	27533
31471	19736	31258	19152	32258	34547	35242	42396	14274	22228
23922	17403	35338	19165	32306	35252	29345	34927	26536	30021

Traspuesta A:

11	85	19	65	10	23	31	62	98	25
84	5	11	10	8	72	81	36	45	16
37	10	45	53	71	97	28	88	10	51
98	35	82	82	75	82	66	33	62	73
11	22	98	87	95	75	56	54	8	93
67	97	54	96	2	53	39	4	2	63
70	57	44	41	92	85	17	94	94	67
13	57	59	62	13	5	75	73	82	71
95	66	89	64	46	96	76	45	12	60
87	80	44	92	62	46	6	28	96	28

Traspuesta B:

87	9	13	40	45	24	49	60	33	100
23	78	90	22	20	30	22	12	15	86
54	30	91	67	34	100	95	7	86	89
20	24	39	18	25	26	40	49	52	62
100	4	35	28	85	73	94	45	34	67
68	33	82	73	25	86	45	85	70	90
66	46	59	73	13	62	77	85	4	74
93	94	54	86	44	21	81	88	47	77
6	54	61	9	96	46	22	38	81	32
49	78	78	61	100	56	51	0	32	34

Inversa A:

-0.46	-0.31	1.23	-1.45	-0.57	0.79	-1.25	0.51	-0.82	0.37
-1.11	0.66	-0.33	-1.38	1.44	-0.90	1.38	0.68	0.38	-0.62
0.21	-0.66	-0.63	-1.23	-0.12	-1.06	1.41	-0.01	-0.08	0.50
0.49	-0.98	-0.17	0.68	-1.39	0.09	-0.66	-0.85	-0.45	0.25
-0.49	-0.63	0.83	-0.70	-1.38	-1.10	0.22	0.71	-0.27	0.55
-0.41	1.02	-0.86	1.05	-0.27	1.45	1.27	-0.79	0.19	1.15
1.24	-1.07	1.29	-0.21	-0.84	0.98	-0.83	-0.71	-1.28	-0.62
0.98	-0.36	-0.76	-0.18	-1.23	-1.16	1.37	0.05	1.12	-0.63
-0.96	-0.42	-0.50	1.13	1.27	-0.11	-1.44	0.72	0.50	1.02
0.46	1.44	-0.54	-1.42	-0.93	-0.04	0.93	0.95	-1.31	1.15

Inversa B:

1.04	-1.47	-1.12	1.44	1.28	-0.25	-1.35	1.07	-0.14	1.12
-1.05	-1.15	-0.19	-0.70	1.17	-0.36	-0.13	0.55	0.59	0.90
-1.30	0.76	-1.24	-1.25	-1.42	-0.31	1.41	-0.52	-0.95	0.83
0.23	0.66	0.65	-0.38	-0.85	0.56	1.45	1.13	-0.30	0.93
-0.35	-0.90	-0.47	-0.16	-0.53	-1.40	1.16	0.42	1.31	-0.05
-1.37	-0.81	1.00	-0.30	-1.06	-0.27	-0.30	0.38	0.39	-0.64
1.38	0.92	0.81	-1.19	0.43	0.46	0.05	-1.16	-0.16	0.92
-0.68	0.25	1.40	1.47	-0.04	0.05	1.06	-0.45	-1.16	-0.59
-0.18	1.47	-0.61	0.03	-0.94	-1.23	1.21	0.26	-0.96	0.43
1.18	-1.06	-1.47	-0.77	-1.22	-0.90	-0.92	-0.99	0.41	0.98

WINDOWS

Padre.c

```
#include <windows.h>
#include <stdio.h>
#include <time.h>
#define N 10
#define TAM_MEM (2 * N * N * sizeof(int))
void LlenarMatriz(int matriz[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = rand() % 101;
        }
    }
}
void ImprimirMatriz(int matriz[N][N]){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }
}
HANDLE CrearMemoriaCompartida(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}
void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}
```

```
void escribirMemoria2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    EscribirMemoriaCompartida(matrizuno, punteroDatos);
    EscribirMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}
int main(int argc, char *argv[]) {
    srand(time(NULL));
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    LlenarMatriz(Matrizuno);
    LlenarMatriz(Matrizdos);
    printf("Matriz A:\n");
    ImprimirMatriz(Matrizuno);
    printf("\nMatriz B:\n");
    ImprimirMatriz(Matrizdos);
    int* apDatos1;
    HANDLE memCom1 = CrearMemoriaCompartida("MemCom1", &apDatos1);
    escribirMemoria2(Matrizuno, Matrizdos, apDatos1);
    HANDLE semRes;
    if ((semRes = CreateSemaphore(NULL, 0, 1, "semRes")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
    STARTUPINFO si[6];
    PROCESS_INFORMATION pi[6];
    HANDLE handles[6];
    for (int j = 1; j <= 6; j++){
        ZeroMemory(&si[j-1], sizeof(si[j-1]));
        si[j-1].cb = sizeof(si[j-1]);
        ZeroMemory(&pi[j-1], sizeof(pi[j-1]));
        escribirMemoria2(Matrizuno, Matrizdos, apDatos1);
        switch (j){
            case 1:
                if (!CreateProcess(NULL, "sum", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
                    printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                    return 1;
                }
                break;
        }
    }
```

```

    case 2:
        if(!CreateProcess(NULL, "res", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
            printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
            return 1;
        }
        break;
    case 3:
        if(!CreateProcess(NULL, "mul", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
            printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
            return 1;
        }
        break;
    case 4:
        if(!CreateProcess(NULL, "tra", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
            printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
            return 1;
        }
        break;
    case 5:
        if(!CreateProcess(NULL, "inv", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
            printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
            return 1;
        }
        break;
    case 6:
        WaitForSingleObject(semRes, INFINITE);
        if(!CreateProcess(NULL, "col", NULL, NULL, FALSE, 0, NULL, NULL, &si[j-1], &pi[j-1])) {
            printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
            return 1;
        }
        break;
}
if (j < 6){
    HANDLE semMat;
    if ((semMat = CreateSemaphore(NULL, 0, 1, "semMat")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
}

```

```

        WaitForSingleObject(semMat, INFINITE);
        CloseHandle(semMat);
    }
    handles[j-1] = pi[j-1].hProcess;
}
UnmapViewOfFile(apDatos1);
CloseHandle(memCom1);
WaitForMultipleObjects(6, handles, TRUE, INFINITE);
for (int j = 0; j < 6; j++) {
    CloseHandle(pi[j].hProcess);
    CloseHandle(pi[j].hThread);
}
return 0;
}

```

Suma.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (2 * N * N * sizeof(int))
#define TAM_MEM2 (5 * N * N * sizeof(int))
void SumarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = matrizuno[i][j] + matrizdos[i][j];
        }
    }
}

int* abrir(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile(hMemCom,FILE_MAP_ALL_ACCESS, 0,0,TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

HANDLE crear(const char* nombre, int** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE,NULL,PAGE_READWRITE,0,TAM_MEM2,nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (int*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM2)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}
```



```

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void EscribirMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    EscribirMemoriaCompartida(matrizuno, punteroDatos);
    EscribirMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

void LeerMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    LeerMemoriaCompartida(matrizuno, punteroDatos);
    LeerMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE memCom1;
    int* apDatos1;
    apDatos1 = abrir("MemCom1", memCom1);
    LeerMemoriaCompartida2(Matrizuno, Matrizdos, apDatos1);
    UnmapViewOfFile(apDatos1);
    CloseHandle(memCom1);
    HANDLE semMat;
    if ((semMat = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semMat")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semMat, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
}

```

```

    int R[N][N];
    SumarMatrices(Matrizuno, Matrizdos, R);
    int* apDatos2;
    HANDLE memCom2 = crear("memComRes", &apDatos2);
    EscribirMemoriaCompartida(R, apDatos2);
    HANDLE semRes;
    if ((semRes = CreateSemaphore(NULL, 0, 1, "semRes1")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
    WaitForSingleObject(semRes, INFINITE);
    UnmapViewOfFile(apDatos2);
    CloseHandle(memCom2);
    printf("Terminado");
    return 0;
}

```

Resta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (2 * N * N * sizeof(int))
void RestarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++) {
            resultado[i][j] = matrizuno[i][j] - matrizdos[i][j];
        }
    }
}

int* abrir(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile( hMemCom,FILE_MAP_ALL_ACCESS, 0,0,TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    punteroDatos += N * N;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}
```

```

void LeerMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    LeerMemoriaCompartida(matrizuno, punteroDatos);
    LeerMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE memCom1;
    int* apDatos1;
    apDatos1 = abrir("MemCom1", memCom1);
    LeerMemoriaCompartida2(Matrizuno, Matrizdos, apDatos1);
    UnmapViewOfFile(apDatos1);
    CloseHandle(memCom1);
    HANDLE semMat;
    if ((semMat = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semMat")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semMat, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    int R[N][N];
    RestarMatrices(Matrizuno, Matrizdos, R);
    HANDLE memCom2;
    int* apDatos2;
    apDatos2 = abrir("memComRes", memCom2);
    EscribirMemoriaCompartida(R, apDatos2);
    HANDLE semRes;
    if ((semRes = CreateSemaphore(NULL, 0, 1, "semRes2")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
    WaitForSingleObject(semRes, INFINITE);
    UnmapViewOfFile(apDatos2);
    CloseHandle(memCom2);
    return 0;
}

```

Multipliación.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (2 * N * N * sizeof(int))
void MultiplicarMatrices(int matrizuno[N][N], int matrizdos[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++){
                resultado[i][j] += matrizuno[i][k] * matrizdos[k][j];
            }
        }
    }
}

int* abrir(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile( hMemCom,FILE_MAP_ALL_ACCESS, 0,0,TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    punteroDatos += 2*(N * N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}
```

```

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

void LeerMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    LeerMemoriaCompartida(matrizuno, punteroDatos);
    LeerMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE memcom1;
    int* apDatos1;
    apDatos1 = abrir("MemCom1", memcom1);
    LeerMemoriaCompartida2(Matrizuno, Matrizdos, apDatos1);
    UnmapViewOfFile(apDatos1);
    CloseHandle(memcom1);
    HANDLE semMat;
    if ((semMat = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semMat")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semMat, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    int R[N][N];
    MultiplicarMatrices(Matrizuno, Matrizdos, R);
    HANDLE memcom2;
    int* apDatos2;
    apDatos2 = abrir("memComRes", memcom2);
    EscribirMemoriaCompartida(R, apDatos2);
    HANDLE semres;
    if ((semres = CreateSemaphore(NULL, 0, 1, "semRes3")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
    WaitForSingleObject(semres, INFINITE);
    UnmapViewOfFile(apDatos2);

```

```

        CloseHandle(memcom2);
        return 0;
    }

```

Transpuesta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (2 * N * N * sizeof(int))
void TransponerMatriz(int matriz[N][N], int resultado[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            resultado[j][i] = matriz[i][j];
        }
    }
}

int* abrir(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile( hMemCom,FILE_MAP_ALL_ACCESS, 0,0,TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}

void EscribirMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    punteroDatos += 3 * (N * N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void LeerMemoriaCompartida(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}
```

```

void LeerMemoriaCompartida2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    LeerMemoriaCompartida(matrizuno, punteroDatos);
    LeerMemoriaCompartida(matrizdos, punteroDatos + (N * N));
}

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE memcom1;
    int* apDatos1;
    apDatos1 = abrir("MemCom1", memcom1);
    LeerMemoriaCompartida2(Matrizuno, Matrizdos, apDatos1);
    UnmapViewOfFile(apDatos1);
    CloseHandle(memcom1);
    HANDLE semmat;
    if ((semmat = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semMat")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semmat, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    int TraspuestaUno[N][N];
    int TraspuestaDos[N][N];
    TransponerMatriz(Matrizuno, TraspuestaUno);
    TransponerMatriz(Matrizdos, TraspuestaDos);
    HANDLE memcom2;
    int* apDatos2;
    apDatos2 = abrir("memComRes", memcom2);
    EscribirMemoriaCompartida(TraspuestaUno, apDatos2);
    EscribirMemoriaCompartida(TraspuestaDos, apDatos2 + (N * N));
    HANDLE sem1;
    if ((sem1 = CreateSemaphore(NULL, 0, 1, "semRes4")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
    WaitForSingleObject(sem1, INFINITE);
    UnmapViewOfFile(apDatos2);
    CloseHandle(memcom2);
    return 0;
}

```

Inversa.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define N 10
#define TAM_MEM (2 * N * N * sizeof(int))
#define TAM_MEM2 (2 * N * N * sizeof(double))
void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++){
        for (int col = 0; col < n; col++){
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1){
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n){
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];
    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++){
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}
```

```
void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1){
        adjunta[0][0] = 1;
        return;
    }
    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

void InversaMatriz(int matriz[N][N], double inversa[N][N]){
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

int* abrir(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE, nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile(hMemCom, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}
```



```

HANDLE crearMemoriaD(const char* nombre, double** punteroDatos){
    HANDLE hMemoria;
    if ((hMemoria = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, TAM_MEM2, nombre)) == NULL) {
        printf("No se creó la memoria compartida: (%i)\n", GetLastError());
        exit(-1);
    }
    if ((*punteroDatos = (double*)MapViewOfFile(hMemoria, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM2)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemoria);
        exit(-1);
    }
    return hMemoria;
}

void escribirMemoria(double matriz[N][N], double* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            *punteroDatos++ = matriz[i][j];
        }
    }
}

void escribirMemoria2(double matrizuno[N][N], double matrizdos[N][N], double* punteroDatos){
    escribirMemoria(matrizuno, punteroDatos);
    escribirMemoria(matrizdos, punteroDatos + (N * N));
}

void leerMemoria(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}

void leerMemoria2(int matrizuno[N][N], int matrizdos[N][N], int* punteroDatos){
    leerMemoria(matrizuno, punteroDatos);
    leerMemoria(matrizdos, punteroDatos + (N * N));
}

```

```

int main(void){
    int Matrizuno[N][N];
    int Matrizdos[N][N];
    HANDLE memcom1;
    int* apDatos1;
    apDatos1 = abrir("MemCom1", memcom1);
    leerMemoria2(Matrizuno, Matrizdos, apDatos1);
    UnmapViewOfFile(apDatos1);
    CloseHandle(memcom1);
    HANDLE semmat;
    if ((semmat = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semMat")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semmat, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    double Inversa_uno[N][N];
    double Inversa_dos[N][N];
    InversaMatriz(Matrizuno, Inversa_uno);
    InversaMatriz(Matrizdos, Inversa_dos);
    double* apDatos2;
    HANDLE hMemComResInversas = crearMemoriaD("memComInv", &apDatos2);
    escribirMemoria2(Inversa_uno, Inversa_dos, apDatos2);
    HANDLE semres;
    if ((semres = OpenSemaphore(SEMAPHORE_ALL_ACCESS, FALSE, "semRes")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semres, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    HANDLE seminv;
    if ((seminv = CreateSemaphore(NULL, 0, 1, "semInv")) == NULL) {
        printf("Falla al invocar CreateSemaphore: %d\n", GetLastError());
        return -1;
    }
    WaitForSingleObject(seminv, INFINITE);
    UnmapViewOfFile(hMemComResInversas);
    CloseHandle(apDatos2);
    return 0;
}

```

Colector.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 10
#define TAM_MEM (5 * N * N * sizeof(int))
#define TAM_MEM2 (2 * N * N * sizeof(double))
void ImprimirMatriz(int matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%d\t", matriz[i][j]);
            printf("\n");
        }
    }
}
void ImprimirInversa(double matriz[N][N]){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            printf("%lf ", matriz[i][j]);
            printf("\n");
        }
    }
}
int* abrir(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile(hMemCom, FILE_MAP_ALL_ACCESS, 0, 0, TAM_MEM)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}
```

```
int* abrir2(const char* nombre, HANDLE hMemCom){
    int* apDatos;
    if ((hMemCom = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,nombre)) == NULL) {
        printf("No se accedió a la memoria compartida para %s: (%i)\n", nombre, GetLastError());
        exit(-1);
    }
    if ((apDatos = (int *)MapViewOfFile(hMemCom,FILE_MAP_ALL_ACCESS, 0,0,TAM_MEM2)) == NULL) {
        printf("No se enlazó la memoria compartida: (%i)\n", GetLastError());
        CloseHandle(hMemCom);
        exit(-1);
    }
    return apDatos;
}
void leer(int matriz[N][N], int* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}
void leer2(int matrizuno[N][N], int matrizdos[N][N], int matriztres[N][N], int matrizcuatro[N][N], int matrizcinco[N][N], int* punteroDatos){
    leer(matrizuno, punteroDatos);
    leer(matrizdos, punteroDatos + (N * N));
    leer(matriztres, punteroDatos + (2 * N * N));
    leer(matrizcuatro, punteroDatos + (3 * N * N));
    leer(matrizcinco, punteroDatos + (4 * N * N));
}
void leerD(double matriz[N][N], double* punteroDatos){
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = *punteroDatos++;
        }
    }
}
void leerD2(double matrizuno[N][N], double matrizdos[N][N], double* punteroDatos){
    leerD(matrizuno, punteroDatos);
    leerD(matrizdos, punteroDatos + (N * N));
}
```

```

int main(void){
    int sum[N][N];
    int res[N][N];
    int mul[N][N];
    int ta[N][N];
    int tb[N][N];
    HANDLE memcom1;
    int* apDatos1;
    abrir("memComRes", memcom1);
    leer2(sum, res, mul, ta, tb, apDatos1);
    double ia[N][N];
    double ib[N][N];
    HANDLE memcom2;
    double* apDatos2;
    abrir2("memComInv", memcom2);
    leerD2(ia, ib, apDatos2);
    printf("\nResultado suma: \n");
    ImprimirMatriz(sum);
    printf("\nResultado resta: \n");
    ImprimirMatriz(res);
    printf("\nResultado multiplicacion: \n");
    ImprimirMatriz(mul);
    printf("\nResultado traspuesta uno: \n");
    ImprimirMatriz(ta);
    printf("\nResultado traspuesta dos: \n");
    ImprimirMatriz(tb);
    printf("\nResultado inversa uno: \n");
    ImprimirInversa(ia);
    printf("\nResultado inversa dos: \n");
    ImprimirInversa(ib);
    HANDLE semres;
    if ((semres = OpenSemaphore(SEMAPHORE_ALL_ACCESS , FALSE, "semRes1")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semres, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
}

```

```

    if ((semres = OpenSemaphore(SEMAPHORE_ALL_ACCESS , FALSE, "semRes2")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semres, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    if ((semres = OpenSemaphore(SEMAPHORE_ALL_ACCESS , FALSE, "semRes3")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semres, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    if ((semres = OpenSemaphore(SEMAPHORE_ALL_ACCESS , FALSE, "semRes4")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semres, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    if ((semres = OpenSemaphore(SEMAPHORE_ALL_ACCESS , FALSE, "semRes5")) == NULL) {
        printf("Falla al invocar OpenSemaphore: %d\n", GetLastError());
        return -1;
    }
    if (!ReleaseSemaphore(semres, 1, NULL)) {
        printf("Falla al invocar ReleaseSemaphore: %d\n", GetLastError());
    }
    UnmapViewOfFile(apDatos1);
    CloseHandle(memcom1);
    UnmapViewOfFile(apDatos2);
    CloseHandle(memcom2);
    return 0;
}

```

```
C:\Users\vbcsl>padre
```

```
Matriz A:
```

23	50	62	26	0	20	100	35	79	99
34	30	24	52	15	68	19	98	81	68
41	88	23	36	61	2	66	35	59	61
26	34	29	10	82	15	9	43	9	93
0	32	53	5	14	0	53	70	89	15
97	21	17	92	53	95	3	73	0	35
62	99	12	52	44	1	95	100	4	21
91	13	19	61	18	82	50	99	52	29
88	91	9	32	58	6	31	52	92	45
37	46	46	84	98	94	67	47	0	100

```
Matriz B:
```

35	1	7	18	72	91	73	96	6	61
39	86	100	63	55	2	62	92	95	63
91	60	3	51	13	50	59	11	33	44
71	96	16	77	59	66	38	71	72	78
68	58	1	70	77	18	14	34	76	52
17	89	69	35	40	94	47	16	68	92
92	10	66	81	47	31	92	49	62	45
87	84	15	42	4	24	66	34	21	55
48	74	51	12	28	96	90	29	12	56
99	59	82	83	85	51	51	96	20	42

```
Resultado suma:
```

58	51	69	44	72	111	173	131	85	160
73	116	124	115	70	70	81	190	176	131
132	148	26	87	74	52	125	46	92	105
97	130	45	87	141	81	47	114	81	171
68	90	54	75	91	18	67	104	165	67
114	110	86	127	93	189	50	89	68	127
154	109	78	133	91	32	187	149	66	66
178	97	34	103	22	106	116	133	73	84
136	165	60	44	86	102	121	81	104	101
136	105	128	167	183	145	118	143	20	142

```
Resultado resta:
```

-12	49	55	8	-72	-71	27	-61	73	38
-5	-56	-76	-11	-40	66	-43	6	-14	5
-50	28	20	-15	48	-48	7	24	26	17
-45	-62	13	-67	23	-51	-29	-28	-63	15
-68	-26	52	-65	-63	-18	39	36	13	-37
80	-68	-52	57	13	1	-44	57	-68	-57
-30	89	-54	-29	-3	-30	3	51	-58	-24
4	-71	4	19	14	58	-16	65	31	-26
40	17	-42	20	30	-90	-59	23	80	-11
-62	-13	-36	1	13	43	16	-49	-20	58

Resultado multiplicacion:

36421	27946	26415	28163	23013	25462	34034	27541	20029	26156
31306	34396	21280	23431	20686	28633	30114	24718	18922	29749
31686	27726	22823	27154	25071	20380	28925	28938	23484	25982
25624	21596	14270	21486	20375	14348	17169	20773	15744	18608
24101	21105	13770	15630	10360	15928	23768	13356	12337	17315
27604	31511	15073	24269	25077	29305	24968	27274	22669	32280
33535	27814	21011	28770	23323	17558	30782	30583	25787	27864
30950	30922	18461	23492	22446	31712	32287	26504	20127	32269
30013	29565	21935	24044	26118	25696	31766	31534	21608	28858
41654	39385	26252	38229	33891	30208	31615	33571	33139	37275

Resultado traspuesta uno:

23	34	41	26	0	97	62	91	88	37
50	30	88	34	32	21	99	13	91	46
62	24	23	29	53	17	12	19	9	46
26	52	36	10	5	92	52	61	32	84
0	15	61	82	14	53	44	18	58	98
20	68	2	15	0	95	1	82	6	94
100	19	66	9	53	3	95	50	31	67
35	98	35	43	70	73	100	99	52	47
79	81	59	9	89	0	4	52	92	0
99	68	61	93	15	35	21	29	45	100

Resultado traspuesta dos:

35	39	91	71	68	17	92	87	48	99
1	86	60	96	58	89	10	84	74	59
7	100	3	16	1	69	66	15	51	82
18	63	51	77	70	35	81	42	12	83
72	55	13	59	77	40	47	4	28	85
91	2	50	66	18	94	31	24	96	51
73	62	59	38	14	47	92	66	90	51
96	92	11	71	34	16	49	34	29	96
6	95	33	72	76	68	62	21	12	20
61	63	44	78	52	92	45	55	56	42

Resultado inversa uno:

1.221172	1.568684	-0.333349	-1.885043	-0.161630	-2.101091	1.101321	-0.186295	1.930523	-1.304011
-0.316259	1.766354	-0.251853	-0.771492	0.213995	-1.241326	-1.890863	1.329620	0.971328	-2.004449
-1.208209	0.456366	1.675261	0.912464	1.412516	1.039776	0.787513	-1.436714	-1.543594	-0.658266
-1.721729	0.075655	-0.636426	0.868366	-0.403120	1.655835	-0.245358	0.222731	1.298074	1.907129
-1.370659	0.352937	0.247063	-0.659736	0.211066	-1.966331	1.732512	0.403841	0.833933	-0.715108
1.349428	-0.983881	-0.315552	-1.677849	1.780761	-2.015483	-2.083375	0.253933	-0.800150	1.855373
1.434386	-1.492778	-1.157714	1.243090	-1.157763	1.196227	1.426966	-0.375810	-0.256189	0.789073
-2.174758	0.955224	0.223075	-0.151225	0.208398	0.067095	1.186811	0.975946	-0.096435	-0.345115
2.127211	-1.412982	0.246132	-0.768117	1.439307	0.306777	2.046279	-1.758943	1.088424	-1.662797
1.002185	-2.064029	1.426909	1.365117	0.193267	0.590768	-0.597995	-0.604707	-1.476877	1.062787

Resultado inversa dos:

-0.396881	0.336269	1.512641	1.043230	-0.443211	0.706019	0.064687	0.284202	0.917109	-0.498404
-1.459857	0.025082	0.988424	-0.130481	0.939083	0.844934	0.098765	-0.643737	-0.922479	-1.086434
0.168640	-0.888212	-0.909340	-0.269257	-0.084442	-1.370412	-0.102102	-1.071703	1.028600	1.429790
-0.052144	-0.718426	1.239640	-0.140652	-0.075205	-1.226013	1.131910	-0.208678	0.194309	-1.406326
-1.493192	-1.333995	-1.305764	-1.072439	0.165412	1.090763	-1.503661	-1.475109	1.282567	0.563205
-0.527953	-0.390415	-1.005444	-0.055764	0.792063	-0.455255	-0.016095	-0.727516	-1.483232	0.081305
1.040279	-1.121537	0.937961	-0.956813	-1.373971	0.090266	0.574817	-1.534948	0.120871	-0.360446
0.456600	1.429188	-1.475925	-1.509848	1.514344	0.927209	-1.508895	0.704467	0.623442	1.341225
1.454460	0.073871	0.596820	-0.942572	-1.248396	-0.779983	-1.030288	0.148949	0.213529	-1.208410
-1.377607	0.752936	0.850292	0.878836	1.532675	0.951024	1.242438	1.029157	0.035305	0.750958

Conclusión

En esta práctica se exploraron los conceptos fundamentales de sincronización de procesos y comunicación interprocesos (IPC) utilizando semáforos y memoria compartida en sistemas operativos Linux y Windows. En Linux, se emplearon funciones como `semget()` y `semop()` para la creación y manipulación de semáforos. `semget()` permite crear o acceder a conjuntos de semáforos mediante una clave única (`key_t`), devolviendo un identificador asociado, mientras que `semop()` facilita la manipulación mediante estructuras `sembuf` que realizan operaciones atómicas como decrementos (`wait`) y aumentos (`signal`). Estos semáforos garantizan exclusión mutua y sincronización, evitando condiciones de carrera en regiones críticas.

En el entorno Windows, se utilizaron funciones como `CreateSemaphore()` y `ReleaseSemaphore()`. `CreateSemaphore()` inicializa un semáforo con un contador inicial y un límite máximo, permitiendo el uso entre procesos; `OpenSemaphore()` facilita el acceso de otros procesos a un semáforo existente, y `ReleaseSemaphore()` incrementa el contador del semáforo, desbloqueando procesos en espera. Aunque similares a los de Linux en funcionalidad, los semáforos en Windows están diseñados para aprovechar las capacidades del entorno WinAPI, ofreciendo sincronización efectiva entre procesos y subprocesos.

Se desarrollaron programas que ejemplifican la sincronización entre un proceso padre y uno hijo. El proceso padre crea e inicializa el semáforo, mientras que el hijo utiliza operaciones de `wait` para acceder a la región crítica y de `signal` para liberarla. Este esquema asegura que solo un proceso acceda a la región crítica en un momento dado, evitando conflictos y garantizando el correcto funcionamiento concurrente.

Adicionalmente, se implementaron soluciones que utilizan memoria compartida para almacenar matrices, limitando el uso a tres regiones de memoria compartida para optimizar recursos. Los semáforos garantizaron la sincronización de las operaciones de lectura y escritura sobre estas regiones, asegurando la coherencia de los datos y evitando condiciones de carrera. Este enfoque integró mecanismos de IPC, como memoria compartida y semáforos, para gestionar eficientemente los recursos en aplicaciones concurrentes.

La práctica demostró la eficacia de los semáforos como mecanismos de sincronización en sistemas operativos Linux y Windows. Su integración con memoria compartida reforzó su utilidad en entornos concurrentes, permitiendo la interacción controlada entre procesos que comparten recursos. Estas herramientas aseguran exclusión mutua, evitan condiciones de carrera y optimizan el uso de memoria y CPU, proporcionando una base sólida para el desarrollo de sistemas paralelos y concurrentes, esenciales en aplicaciones robustas y eficientes en entornos multitarea.